C++ Quick Reference and Debugging Examples

Vectors and bounds

```
#include <vector>
std::vector<int> values{10, 20, 30};
for (std::size_t i = 0; i < values.size(); ++i) {
    std::cout << values[i] << '\n';
}
```

Valid vector indexes are 0 through size() - 1. Use i < values.size(), not <=. at(i) performs bounds checking and throws std::out_of_range when invalid.

Range-based loops

```
for (int value : values) {
    std::cout << value << '\n';
}
```

A range-based for loop avoids manual index errors when the index is not needed. Use const auto& value to avoid copying larger elements.

Sorting

```
#include <algorithm>
std::sort(values.begin(), values.end());
```

std::sort rearranges the range in place and returns void. Do not assign its result back to the vector.

References and pointers

```
int value = 7;
int& ref = value;
int* ptr = &value;
```

A reference must be initialized and cannot be reseated. A pointer can be null; check it before dereferencing.

Functions and const

```
int sum(const std::vector<int>& values) {
    int total = 0;
    for (int value : values) total += value;
    return total;
}
```

Passing a vector by const reference avoids a copy and prevents accidental modification.